

Slides for P3969R0 — Fixing `std::bit_cast` of types with padding bits

Document number:

P4038R0

Date:

2026-03-10

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/bit-cast-padding-slides.cow



IMPORTANT: This document has custom controls:

-  ,  : go to the next slide
-  ,  : go to previous slide

Fixing `std::bit_cast` of types with padding bits P3969R0

The problem

```
// GCC accepts (x == 0)  
// Clang rejects  
constexpr auto x = std::bit_cast<__int128>(0.0L);
```

- UB due to padding bits in 80-bit x87 `long double`
- this is (arguably) undiagnosed library UB
- `std::bit_cast<__int128, long double> = std::unreachable`
- it would be useful if it worked (for math function implementations)

The problem, cont.

```
struct alignas(4) E { };  
// GCC and Clang error:  
constexpr auto x = std::bit_cast<int>(E{});  
// MSVC accepts:  
static_assert(x == 0);
```

- MSVC treats padding in the original as zero
- see <https://developercommunity.visualstudio.com/t/11027496>
- maybe standardize the ~~bug~~ feature?

Possible solutions

"Mathematically correct" functions eliminate special cases and UB pitfalls:

Single-function	Two-function
<pre>struct alignas(4) E { }; auto x = bit_cast<int>(E{}); assert(x == 0); // OK</pre>	<pre>struct alignas(4) E { }; auto x = bit_cast<int>(E{}); // error auto y = bit_cast_zero_padding<int>(E{}); assert(x == 0); // OK</pre>
✓ already implemented (accidentally) ✓ retroactively fixes code ✓ safety by default ✗ doesn't express intent ✗ not really “bit-casting” ✗ hidden cost, no-opt out	✓ expresses intent ✓ can opt out of cost ✓ no semantic change between versions ✗ more complex ✗ unproven ✗ unimplemented

Gotcha: clearing padding in unions

```
struct alignas(4) E { };
union U {
    int x;
    E e;
};
auto x = bit_cast_zero_padding<int>(U{ .e = {} });
```

- U has padding only when E is active
- cannot clear padding without knowing active alternative
- this discussion is missing from the paper

References

[P3969R0] *Jan Schultke*. Fixing `std::bit_cast` of types with padding bits (2026-10-03)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3969r0.html>